# Homography & Perspective Warp

Week 11 — Rectifying a photographed page (DocScan core)

Practical Image Processing

Lecture 11 ▪ 60–70 min

# Where we are

- Wk10: match the same points across two views of a scene.
- Today: turn those matches into a **geometric transform** that *warps* one image onto the other.
- The payoff: take a photo of a meter/document snapped at an angle and **rectify** it to a flat, front-on scan.

**This is the DocScan core**

Detect page → find its 4 corners (or match features) → estimate a **homography** → **warp** to a clean rectangle.

# Roadmap

# Geometric transforms

# What is a geometric transform?

- So far we changed pixel *values* (brightness, edges). Now we change pixel *positions*.
- A geometric transform maps each output pixel $(x', y')$ back to a source location $(x, y)$.
- Examples: translate, rotate, scale, shear, and — the big one — **perspective**.

**Key idea**

Value operations answer "what colour is here?" Geometric operations answer "where does this point go?"

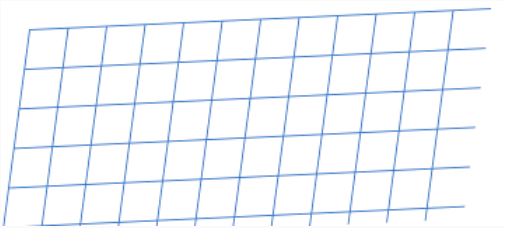
# A hierarchy of 2D transforms

| Transform | Preserves | DoF |
|---|---|---|
| Rigid (rotate+translate) | lengths, angles | 3 |
| Similarity (+scale) | angles, ratios | 4 |
| Affine | parallelism | 6 |
| Projective (homography) | straight lines only | 8 |

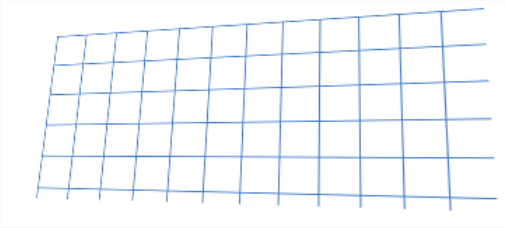
**Key idea**

Each row is more general. A photo taken at an angle needs the most general one: a **homography**.

# Affine vs perspective — on a grid



affine: parallel lines stay parallel



perspective: parallel lines converge

The converging lines are the “vanishing point” effect of a real camera. Only a homography models it.

# Affine transform

- 6 degrees of freedom: a $2 \times 3$ matrix. Maps parallelograms to parallelograms.
- `cv2.getAffineTransform(3 pts, 3 pts)`, then `cv2.warpAffine`.
- Good for rotation/scale/shear — but *cannot* model a page tilted away from the camera.

**Math**

Affine: $\begin{pmatrix} x' \\ y' \end{pmatrix} = A \begin{pmatrix} x \\ y \end{pmatrix} + t$. Straight, parallel lines stay parallel.

# Homography

# The homography

- A **homography** $H$ is a $3 \times 3$ matrix relating two views of a *planar* surface.
- 8 degrees of freedom (the 9th is a scale). Maps straight lines to straight lines, but parallel lines may converge.
- Exactly the model for photographing a flat page/meter from any angle.

**Math**

Using homogeneous coords: $\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = H \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$, then divide by $w'$ to get the pixel $(x'/w', y'/w')$.

# Why divide by $w'$? (the perspective part)

- That final division is what makes far things smaller and parallel lines converge.
- Affine has the bottom row $(0\ 0\ 1) \Rightarrow w' = 1$ always $\Rightarrow$ no perspective.
- Homography's bottom row $(h_{31}\ h_{32}\ 1)$ makes $w'$ depend on position $\Rightarrow$ perspective.

**Key idea**

The nonlinear "divide by $w'$" is the whole difference between affine and perspective.

# How many points define a homography?

- 8 unknowns $\Rightarrow$ need 8 equations $\Rightarrow$ **4 point correspondences** (each gives 2).
- `cv2.getPerspectiveTransform(4 src, 4 dst)` for exactly 4 points.
- With many noisy matches: `cv2.findHomography(pts, pts, cv2.RANSAC)`.

**✓ Good practice**

4 good corners is all you need. For DocScan, those 4 are the document's corners.

# Rectification: the DocScan core

# The rectification idea

1. Find the 4 corners of the page/meter in the photo (Week 9's `approxPolyDP`, or feature matching).
2. Decide where those corners *should* go: the 4 corners of a clean rectangle.
3. Compute the homography from photo-corners $\rightarrow$ rectangle-corners.
4. `warpPerspective` the photo through it: out comes a flat, front-on scan.

# Step 1–2: a tilted meter and its 4 corners



the clean template (target)



simulated angled photo; 4 corners marked (red)

The corners are the correspondences: where each red dot *is* vs where it *should* be.

# Step 3–4: warp to a flat scan



input: angled photo



output: rectified, front-on

`H = getPerspectiveTransform(corners, rectangle)`; `warpPerspective(photo, H, size)`.
Digits are now readable.

# Getting the 4 corners in the right order

- `approxPolyDP` gives 4 points but in *arbitrary* order.
- Order them: top-left, top-right, bottom-right, bottom-left — else the warp flips/twists.
- Trick: TL has the smallest $x+y$; BR the largest; TR the smallest $x-y$; BL the largest.

**✕ Pitfall**

Wrong corner order = a mirrored or rotated scan. Always sort the corners consistently.

# Automatic alignment with RANSAC

# From matches to a homography

- Instead of hand-picking 4 corners, use Week 10's feature matches.
- But matches contain wrong ones — a plain fit would be ruined by outliers.
- **RANSAC**: repeatedly fit $H$ from 4 random matches, keep the $H$ that most matches agree with.

**Math**

RANSAC = RANdom SAmple Consensus. It finds the transform supported by the largest set of **inliers**.

# RANSAC, step by step

1. Pick 4 random correspondences; compute a candidate $H$.
2. Count **inliers**: matches whose points $H$ maps to within a few pixels.
3. Repeat many times; keep the $H$ with the most inliers.
4. Refit $H$ using *all* its inliers for a precise result.

**Key idea**

RANSAC tolerates a large fraction of wrong matches — it only needs 4 good ones to exist.

# Automatic rectification on the real meter



angled photo (input)



auto-rectified via ORB + RANSAC

No manual corners: 66 ORB matches $\rightarrow$ 57 RANSAC inliers $\rightarrow$ `findHomography` $\rightarrow$ flat scan.

# In-class exercise

# In-class exercise (10 min)

**Design the rectifier**

1. Why can't an affine transform straighten a page photographed from an angle?
2. You have the 4 page corners in the photo. What are the destination points for an A4-shaped scan of width 800?
3. Your automatic pipeline has 200 feature matches, 60 of them wrong. Which step saves you?

# Debrief

- (1) Affine keeps parallel lines parallel; a real photo makes them converge — needs the "divide by $w'$".
- (2) $(0,0), (800,0), (800,1131), (0,1131)$ — A4 ratio $\approx$ 1:1.414, so height $= 800 \times 1.414$.
- (3) **RANSAC** inside `findHomography` — it ignores the 60 outliers and fits to the inliers.

**Key idea**

Corners (or matches) + destination rectangle + homography + warp = a scanner in four lines of code.

# Deep dive

# Interpolation: filling the output pixels

- Warping maps output pixels to *fractional* source locations — between pixels.
- **Nearest** (blocky, fast), **bilinear** (smooth, default), **bicubic** (sharper, slower).
- Use `cv2.INTER_LINEAR` for images, `INTER_NEAREST` for label/mask maps.

**Key idea**

Geometry says *where*; interpolation decides the *value* at that fractional spot.

# Planar assumption & its limits

- A homography is exact only for a *flat* surface (or a camera that only rotates).
- A curved page, a bent meter cover, or 3D objects break the model.
- For a gently curved document you need **dewarping** (a nonlinear grid) — beyond a single $H$.

**✕ Pitfall**

If your "scan" still bows at the edges, the surface wasn't flat — one homography can't fix curvature.

# Other uses of homography

- **Panorama stitching**: warp each photo into a common frame and blend.
- **Augmented reality**: place a graphic onto a detected planar marker.
- **Bird's-eye view**: rectify a road/parking scene to a top-down map.
- **DocScan**: our meter/plate rectifier.

# The whole rectifier in code

```
# corners: 4 page corners found in the photo (ordered TL,TR,BR,BL)
W, H = 800, int(800*1.414)
dst = np.float32([[0,0],[W,0],[W,H],[0,H]])
M = cv2.getPerspectiveTransform(corners, dst)
scan = cv2.warpPerspective(photo, M, (W, H))
# automatic variant: ORB matches -> cv2.findHomography(..., cv2.RANSAC)
```

Four lines turn an angled photo into a flat scan — the heart of DocScan.

# Wrap-up

# What you should now be able to do

- Place a transform in the hierarchy (rigid $\rightarrow$ affine $\rightarrow$ projective) and pick the right one.
- Explain a homography and why 4 correspondences define it.
- Rectify a tilted document: order corners, build $H$, `warpPerspective`.
- Align automatically from feature matches with `findHomography` + RANSAC.

# This week's deliverable

**Due before Week 12 (see `project/`)**

1. Take/simulate an angled photo of a meter or document; detect its 4 corners.
2. Order the corners and rectify to a flat scan with `warpPerspective`.
3. Also rectify automatically via ORB matches + `findHomography(RANSAC)`; report inliers.
4. Reflect: after rectification, why is digit reading (Weeks 12–14) much easier?

## A homography turns a snapshot into a scan.

**Four corners, one $3 \times 3$ matrix, one warp — and the page lies flat.**